

Tarea
Desarrollo Web en Entorno Servidor

Unidad 7: Aplicaciones Web Dinámicas

Valoración de Productos

Abraham Quintana Micó 3º DAW Semi B

Índice

Tarea Desarrollo Web en Entorno Servidor.....	1
Unidad 7: Aplicaciones Web Dinámicas.....	1
Valoración de Productos.....	1
Introducción.....	3
Estructura del proyecto.....	4
Desarrollo de la aplicación.....	5
Página de acceso: login.php.....	5
Página principal: listado.php.....	9
Proceso de votación: proceso_voto.php.....	10
Cierre de sesión: proceso_logout.php.....	13
Conexión a la base de datos: conexion_bbdd.php.....	13
Mejoras aportadas.....	13
Conclusión.....	16
Bibliografía.....	16

Introducción

En esta tarea he desarrollado una aplicación web que permite a los usuarios registrados valorar productos en tiempo real.

La aplicación está construida con PHP 8 y JavaScript (que está incluido en el propio enunciado del ejercicio), utilizando “fetch()” para la comunicación asíncrona con el servidor.

He descartado el uso de la librería “Xajax” por incompatibilidad con PHP 8, ya que utiliza funciones obsoletas que impiden la carga de la página.

En su lugar, he optado por una solución moderna y funcional, “fetch()”. Añadiendo “EventListeners” para controlar los cambios en los formularios de las páginas realizados por el usuario y, obteniendo del propio documento los elementos que deseo modificar sin recargar nada. Esto me permite validar formularios y actualizar los datos de la página, en este caso el login o valoraciones sin que se recargue la misma.

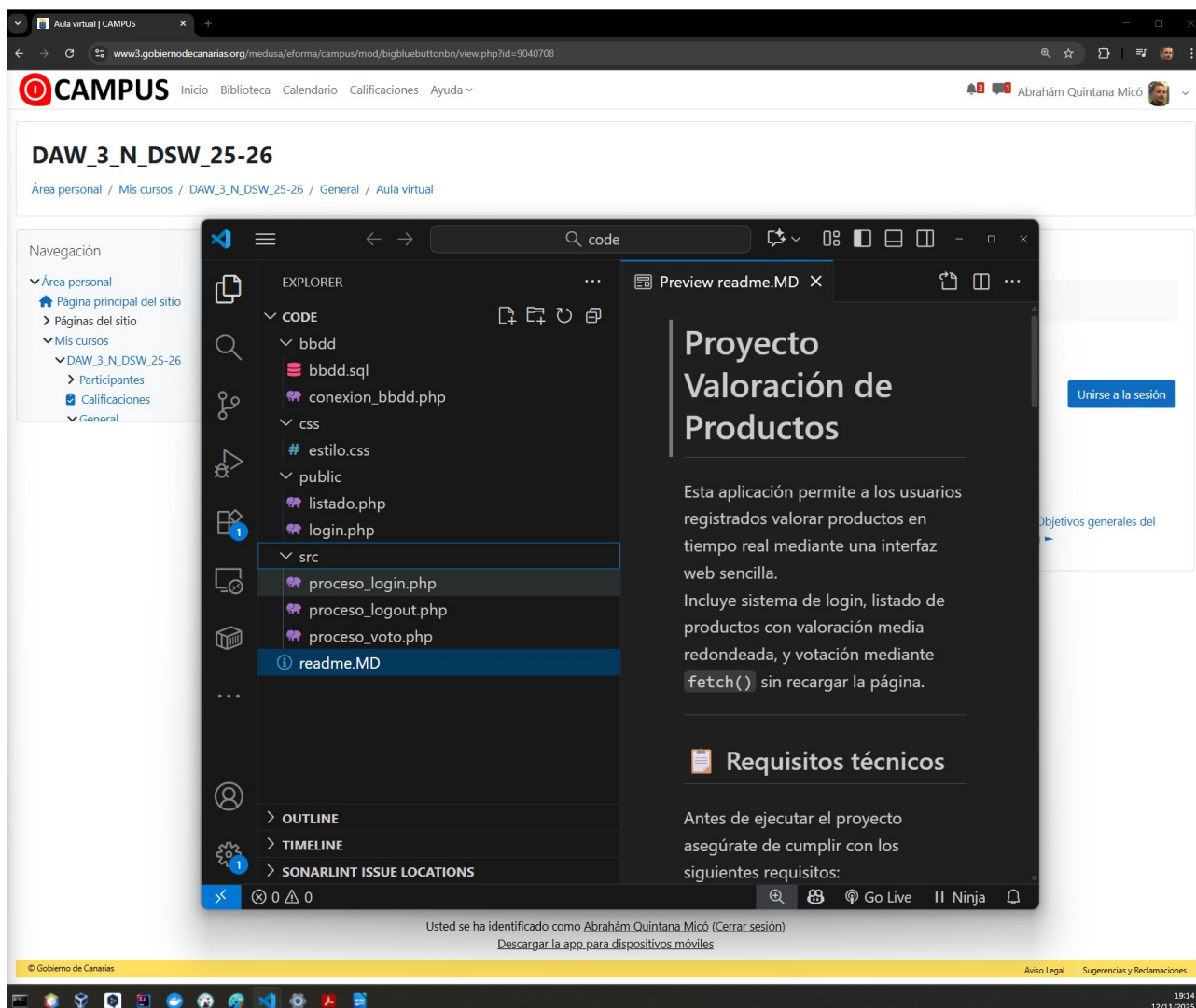
Al usar directamente la función “fetch()” de JavaScript, no es necesario instalar nada con Composer, por lo que la página queda abstraída de cualquier incompatibilidad debida a las dependencias.

Nota: Es necesario importar y ejecutar en phpMyAdmin el fichero que se encuentra en code/bbdd/bbdd.sql para que la aplicación funcione.

Estructura del proyecto

La estructura del proyecto se ha organizado en carpetas separadas para mantener la claridad y facilitar el mantenimiento usando buenas prácticas:

- **/bbdd:** Contiene el script bbdd.sql con la estructura de la base de datos y los datos iniciales, y el archivo conexion_bbdd.php para la conexión con PDO.
- **/css:** Incluye la hoja de estilos estilo.css, adaptada desde la práctica de la UT3, eliminando elementos innecesarios.
- **/public:** Contiene las páginas visibles login.php y listado.php.
- **/src:** Incluye los procesos internos proceso_login.php, proceso_logout.php y proceso_voto.php.
- **Readme.MD:** Archivo readme generado para incluirlo en mi repositorio personal de github, de manera que cuando quiera volver usar o mejorar esta aplicación en el futuro sepa que hace, como funciona, su estructura y que requisitos tiene para su ejecución.



Desarrollo de la aplicación

Página de acceso: login.php

La aplicación comienza en la página login.php, que actúa como entrada para los usuarios.

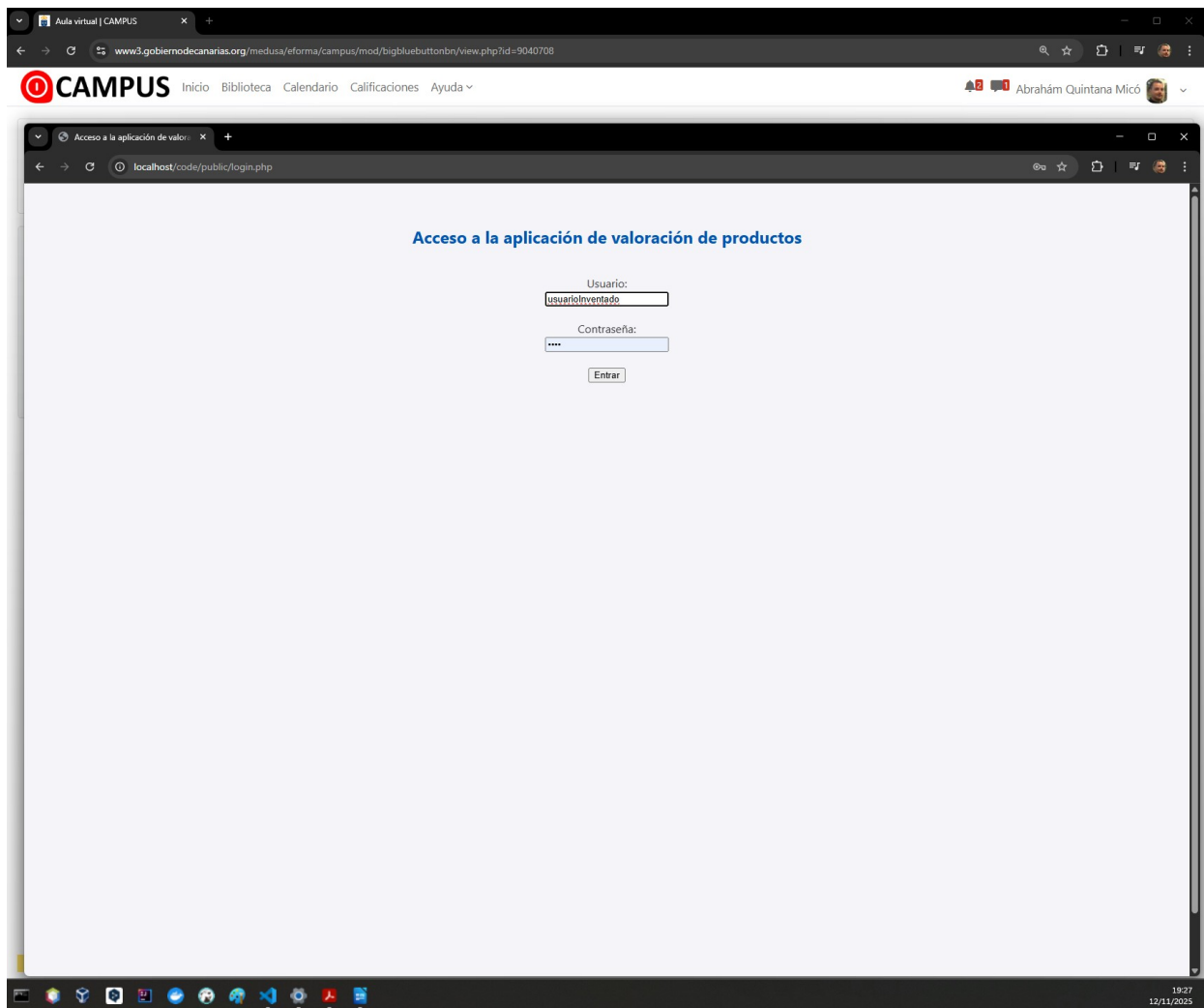
Si se intenta acceder a listado.php sin haber iniciado sesión, el sistema redirige automáticamente a esta página de login. Esto se controla mediante una comprobación de sesión en la cabecera de listado.php, lo que garantiza que solo los usuarios autenticados puedan acceder al contenido protegido.

El formulario de acceso solicita usuario y contraseña.

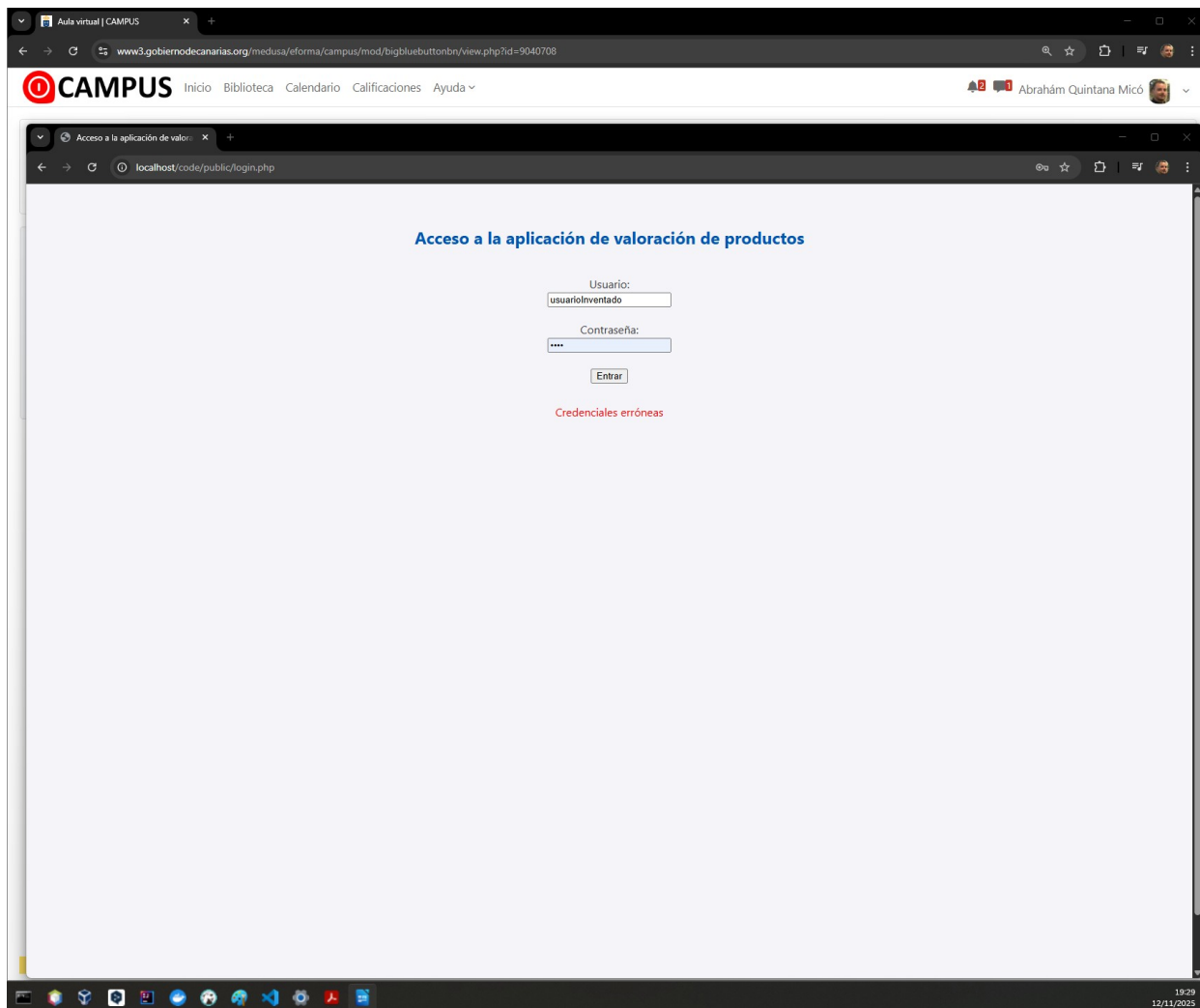
Al enviarlo, con un EventListener, se intercepta el evento submit y, se evita la recarga de la página. Los datos se envían al servidor utilizando fetch() y se procesan en el archivo proceso_login.php. Esta decisión técnica fue necesaria tras descartar el uso de la librería Xajax, que no es compatible con PHP 8 (o al menos a mi me daba un error y tuve que descartar su uso). Al intentar usarla, la página no cargaba debido al uso de funciones obsoletas, por lo que opté por una solución moderna y funcional con fetch().

Si las credenciales son correctas, se inicia la sesión y se redirige automáticamente al listado de productos. En caso contrario, se muestra un mensaje de error en la misma página sin recargar, para que el usuario vea el error y vuelva a intentarlo revisando los datos.

- Login con usuario inexistente antes de pulsar “Entrar”:



- Se muestra el error al pulsar “Entrar”:



- Login con usuario de pruebas redirige automáticamente a la página listado.php
 - Usuario: ana
 - Contraseña: ana123:

Lista de productos disponibles

Usuario conectado: ana

[Cerrar sesión](#)

Nombre	Precio	Valoración/Votos	Votar
Nintendo 3DS negro	270.00 €	3/2	Votar
Acer AX3950 I5-650 4GB 1TB W7HP	410.00 €	Sin valorar	Votar
Archos Clipper MP3 2GB negro	26.70 €	Sin valorar	Votar
Sony Bravia 32IN FULLHD KDL-32BX400	356.90 €	Sin valorar	Votar
Asus EEEPC 1005PKD N455 1 250 BL	245.40 €	Sin valorar	Votar
HP Mini 110-3120 10.1LED N455 1GB 250GB W7S negro	270.00 €	Sin valorar	Votar
Canon Ixus 115HS azul	196.70 €	Sin valorar	Votar
Kingston DataTraveler 16GB DT101G2 USB2.0 negro	19.20 €	Sin valorar	Votar
Kingston DataTraveler G3 32GB rojo	40.00 €	Sin valorar	Votar
Kingston MicroSDHC 8GB	10.20 €	Sin valorar	Votar
Canon Legria FS306 plata	175.00 €	Sin valorar	Votar
LG TDT HD 23 M237WDP-PC FULL HD	186.00 €	Sin valorar	Votar
HP Laserjet Pro Wifi P1102W	99.90 €	Sin valorar	Votar
Pentax Optio LS1100	104.80 €	Sin valorar	Votar
Lector ebooks Papyre6 con SD2GB + 500 ebooks	205.50 €	Sin valorar	Votar
Packard Bell UB103 23.13-550.4G 640GB NV/D/AG210	761.80 €	Sin valorar	Votar

Página principal: listado.php

Una vez que el usuario se valida correctamente en el login, accede a la página listado.php.

Aquí controlo que si alguien intenta entrar directamente sin haber iniciado sesión, se le redirige automáticamente a login.php, comprobando la variable de sesión al inicio del fichero.

En esta página muestro una tabla con todos los productos disponibles. Cada fila contiene el nombre del producto, su precio y la valoración media junto con el número de votos. Para obtener estos datos preparo una consulta SQL que hace un LEFT JOIN con la tabla de votos, de forma que se muestran también los productos que todavía no han sido valorados. En esos casos, en la celda aparece el texto “Sin valorar” como especifica el enunciado.

Si tiene votos, obtengo la media aritmética directamente con la función de sql “Avg” y, para redondear el resultado sin decimales uso “Round”, así los datos obtenidos no tengo que tratarlos con código antes de mostrarlos.

En la tabla-listado, cada producto tiene un select con valores del 1 al 5 para que el usuario pueda votar y, al cambiar el valor del select, se llama a la función votarProducto() en JavaScript, que llama usando fetch() a proceso_voto.php(servidor) con un post y, recibe un JSON con el resultado.

Si el voto se inserta correctamente, actualizo la celda de la tabla en tiempo real con la nueva media y el número de votos. Estos datos vienen en el resultado que devuelve proceso_voto.

Si el usuario ya había votado ese producto, muestro un alert indicando que no puede volver a valorarlo.

De esta forma consigo que las valoraciones se actualicen sin recargar la página, cumpliendo con el requisito de interacción dinámica.

Proceso de votación: proceso_voto.php

El fichero proceso_voto.php es el que recibe la petición de voto desde listado.php. Primero compruebo que la sesión está activa y que se han recibido correctamente los datos del usuario, producto y la puntuación.

Si falta alguno, devuelvo un JSON con success=false y un mensaje de error.

Después verifico si el usuario ya ha votado ese producto. Para ello hago una consulta a la tabla votos buscando por el id del producto y el usuario. Si encuentro un registro, devuelvo un JSON indicando que ya ha valorado ese producto y no permito que se inserte de nuevo.

Si no existe un voto previo, inserto el nuevo registro en la tabla votos. A continuación recalculo la media y el número de votos con una consulta SQL que usa ROUND(AVG(cantidad), 0) y COUNT(*). Devuelvo estos datos en formato JSON para que el front_end actualice la tabla en tiempo real.

He añadido un bloque try-catch para capturar posibles errores durante el desarrollo y devolverlos también en JSON, evitando que se rompa la comunicación con el cliente.

Con este proceso garantizo que cada usuario solo pueda votar una vez por producto y que las valoraciones se reflejen inmediatamente en la interfaz.

- Resultado al intentar votar por segunda vez un producto, muestra el alert con el error y no se tiene en cuenta el voto.:

CAMPUS

[Inicio](#) [Biblioteca](#) [Calendario](#) [Calificaciones](#) [Ayuda](#)

Abrahám Quintana Micó

Listado de productos

localhost/code/public/listado.php

localhost dice

Ya has valorado este producto

Aceptar

Usuario conectado: ana

[Cerrar sesión](#)

Nombre	Precio	Valoración/Votos	Votar
Nintendo 3DS negro	270.00 €	3/2	3
Acer AX3950 I5-650 4GB 1TB W7HP	410.00 €	Sin valorar	--
Archos Clipper MP3 2GB negro	26.70 €	Sin valorar	--
Sony Bravia 32IN FULLHD KDL-32BX400	356.90 €	Sin valorar	--
Asus EEEPC 1005PXD N455 1 250 BL	245.40 €	Sin valorar	--
HP Mini 110-3120 10.1LED N455 1GB 250GB W75 negro	270.00 €	Sin valorar	--
Canon Ixus 115HS azul	196.70 €	Sin valorar	--
Kingston DataTraveler 16GB DT101G2 USB2.0 negro	19.20 €	Sin valorar	--
Kingston DataTraveler G3 32GB rojo	40.00 €	Sin valorar	--
Kingston MicroSDHC 8GB	10.20 €	Sin valorar	--
Canon Legria FS306 plata	175.00 €	Sin valorar	--
LG TDT HD 23 M237WDP-PC FULL HD	186.00 €	Sin valorar	--
HP Laserjet Pro Wifi P1102W	99.90 €	Sin valorar	--
Pentax Optio LS1100	104.80 €	Sin valorar	--
Lector ebooks Papyré6 con SD2GB + 500 ebooks	205.50 €	Sin valorar	--
Parkard Reil IR103 23 I3-550 4G 640GB NVIDIA G210	761.80 €	Sin valorar	--

19:49

12/11/2025

- Resultado al votar por un producto al que ese usuario había votado, en este caso “Archos Clipper MP3 2GB negro ” con valor 5.

Listado de productos disponibles

Usuario conectado: ana

[Cerrar sesión](#)

Nombre	Precio	Valoración/Votos	Votar
Nintendo 3DS negro	270.00 €	3/2	3
Acer AX3950 i5-650 4GB 1TB W7HP	410.00 €	Sin valorar	--
Archos Clipper MP3 2GB negro	26.70 €	5/1	5
Sony Bravia 32IN FULLHD KDL-32BX400	356.90 €	Sin valorar	--
Asus EEEPC 1005PXD N455 1 250 BL	245.40 €	Sin valorar	--
HP Mini 110-3120 10.1LED N455 1GB 250GB W7S negro	270.00 €	Sin valorar	--
Canon Ixus 115HS azul	196.70 €	Sin valorar	--
Kingston DataTraveler 16GB DT101G2 USB2.0 negro	19.20 €	Sin valorar	--
Kingston DataTraveler G3 32GB rojo	40.00 €	Sin valorar	--
Kingston MicroSDHC 8GB	10.20 €	Sin valorar	--
Canon Legria FS306 plata	175.00 €	Sin valorar	--
LG TDT HD 23 M237WDP-PC FULL HD	186.00 €	Sin valorar	--
HP Laserjet Pro Wifi P1102W	99.90 €	Sin valorar	--
Pentax Optio LS1100	104.80 €	Sin valorar	--
Lector ebooks Papyre6 con SD2GB + 500 ebooks	205.50 €	Sin valorar	--
Packard Bell J8103 23.13-550 4G 640GB NVIDIA G210	761.80 €	Sin valorar	--

Cierre de sesión: proceso_logout.php

Para completar el ciclo de autenticación, he creado el fichero proceso_logout.php. En este archivo destruyo la sesión activa y redirijo al usuario de nuevo a login.php. De esta forma, garantizo que al cerrar sesión no quede ninguna variable de sesión pendiente y que el acceso a listado.php vuelva a estar inaccesible.

El enlace de “Cerrar sesión” aparece en la parte superior de listado.php y llama directamente a este script. Con esto consigo que el usuario pueda salir de la aplicación de manera sencilla y segura.

Conexión a la base de datos: conexion_bbdd.php

Para centralizar la conexión a la base de datos, he creado el fichero conexion_bbdd.php. En este defino la conexión mediante PDO, activando el modo de errores con excepciones (PDO::ERRMODE_EXCEPTION). Esto me permite capturar cualquier fallo en las consultas y manejarlo de forma controlada.

He optado por usar PDO en lugar de mysqli porque me ofrece mayor flexibilidad y seguridad, además de permitir consultas preparadas con parámetros. De esta manera evito problemas de inyección SQL y mantengo el código más limpio igual que en la prácticas anteriores.

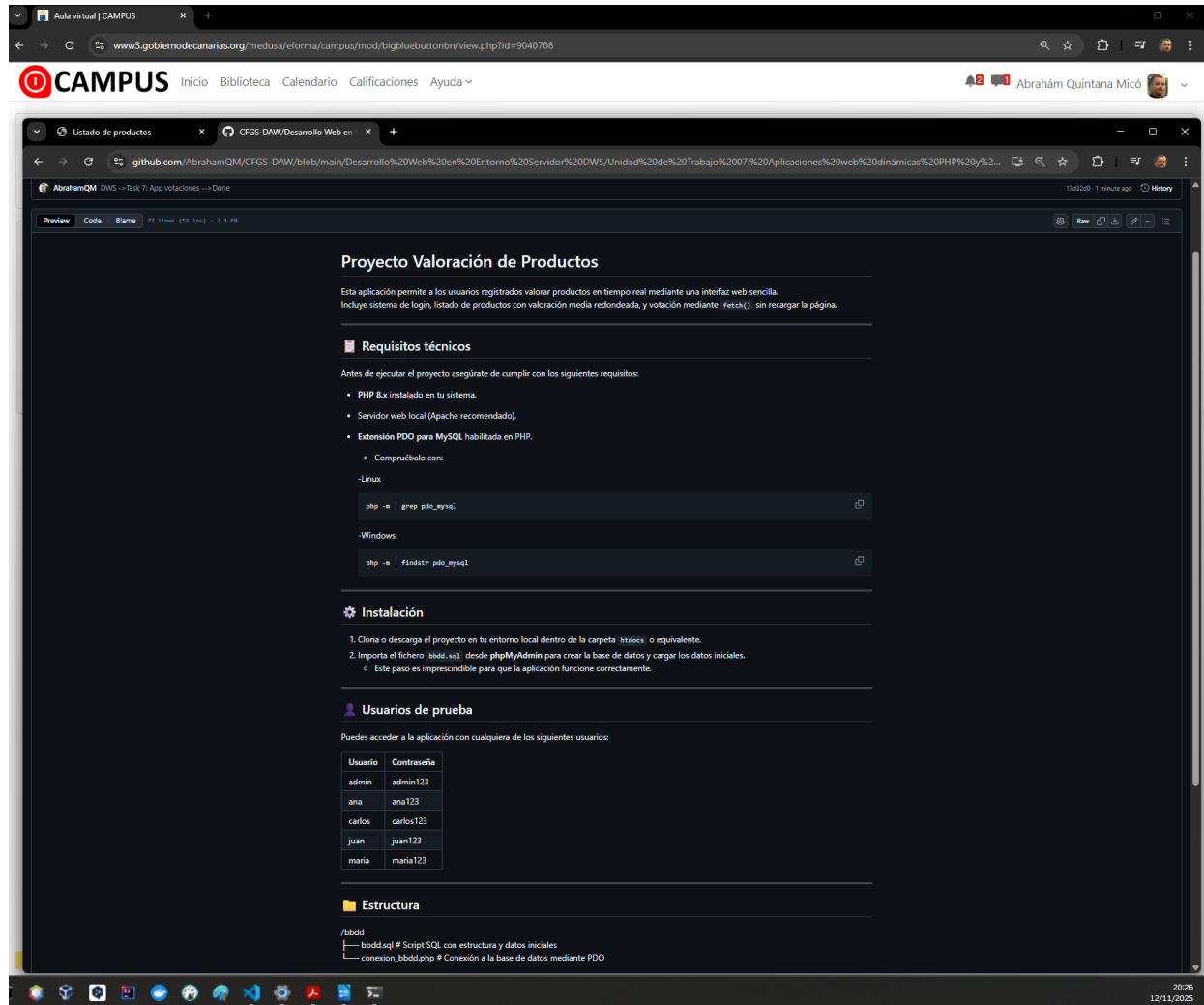
Este archivo se incluye en los procesos y la página de listado (proceso_login.php, proceso_voto.php, listado.php) para no repetir la lógica de conexión en cada uno.

Mejoras aportadas

En esta práctica he añadido varias mejoras respecto al enunciado original:

- He descartado el uso de Xajax por incompatibilidad con PHP 8 y he optado por fetch(), que me ha permitido mantener la funcionalidad de validación y votación en tiempo real sin dependencias obsoletas.
- He organizado la aplicación en carpetas (bbdd, css, public, src) para mantener el código más limpio y fácil de mantener.
- He aprovechado el CSS de la práctica de la UT3, eliminando lo innecesario y adaptándolo a esta aplicación, para darle estilos a la página
- He implementado control de errores en las respuestas del servidor, devolviendo siempre JSON válido para evitar problemas en el cliente.
- He documentado cada archivo con comentarios explicativos, lo que facilita la comprensión del código.
- He añadido un fichero README.md con instrucciones claras de instalación y uso, incluyendo los usuarios de prueba para acceder a la aplicación.

- **Visualización del fichero readme en mi repositorio de github:**



Aula virtual | CAMPUS

www3.gobiernodecanarias.org/medusa/eforma/campus/mod/bigbluebuttonbn/view.php?id=9040708

CAMPUS

Inicio Biblioteca Calendario Calificaciones Ayuda

Abrahám Quintana Micó

Listado de productos

CFGS-DAW / Desarrollo Web en Entorno Servidor DWS / Unidad de Trabajo 07: Aplicaciones web dinámicas PHP y Javascript / Tarea / code / readme.MD

77 lines (51 loc) · 2.1 KB

```

php -e | grep pdo_mysql

~Windows
php -e | findstr pdo_mysql

```

Instalación

1. Clona o descarga el proyecto en tu entorno local dentro de la carpeta `htdocs` o equivalente.
2. Importa el fichero `bbdd.sql` desde phpMyAdmin para crear la base de datos y cargar los datos iniciales.
 - Este paso es imprescindible para que la aplicación funcione correctamente.

Usuarios de prueba

Puedes acceder a la aplicación con cualquiera de los siguientes usuarios:

Usuario	Contraseña
admin	admin123
ana	ana123
carlos	carlos123
juan	juan123
maria	maria123

Estructura

```

/bbdd
├── bbdd.sql # Script SQL con estructura y datos iniciales
└── conexion_bbdd.php # Conexión a la base de datos mediante PDO

/css
├── estilo.css # Hoja de estilos (adaptada desde la práctica UT3)

/public
├── login.php # Página de acceso con validación por fetch()
├── listado.php # Listado de productos con votación en tiempo real

/src
├── proceso_login.php # Procesa el login y crea la sesión
├── proceso_logout.php # Cierra la sesión del usuario
└── proceso_voto.php # Inserta el voto y devuelve media y número de votos en JSON

```

Ejecución del proyecto

Accede a la aplicación desde:

<http://localhost/code/public/login.php>

20:26

12/11/2023

Conclusión

Con esta práctica he cumplido todos los requisitos del enunciado, adaptando la solución a las limitaciones reales del entorno (PHP 8).

La aplicación funciona correctamente: permite validar usuarios, acceder al listado de productos, votar en tiempo real y evita la doble votación.

La estructura es clara, el código está documentado y las mejoras que he aportado facilitan tanto el mantenimiento como la experiencia de usuario.

En definitiva, he conseguido una aplicación completa de valoración de productos que refleja lo aprendido en la unidad y que se ajusta a los criterios de evaluación de la tarea.

Bibliografía

No he utilizado fuentes externas para la realización de esta práctica.

Me he basado en los materiales del campus, en las prácticas anteriores y en las prácticas desarrolladas el año pasado en “Desarrollo Web en Entorno Cliente DEW” para el uso de fetch, EventListeners y la actualización de elementos del documento que se muestra en el navegador en tiempo real sin recargar la página.

Si que he visitado varios foros en StackOverFlow relacionados con el problema de las dependencias.